The Basics

An important part of building a robot is the incorporation of sensors. Sensors translate between the physical world and the abstract world of Microcontrollers. This month in Basics, I will explain the common types of sensors used in personal robotics.

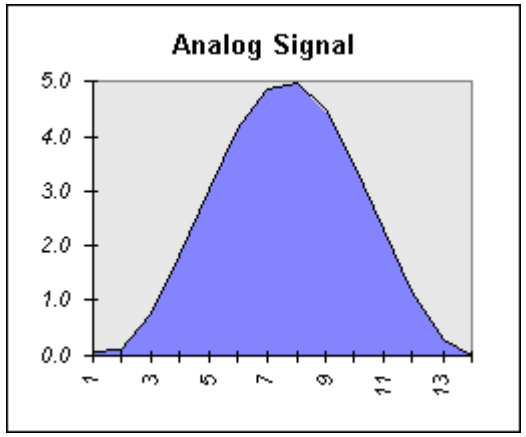
Sensor Output Values

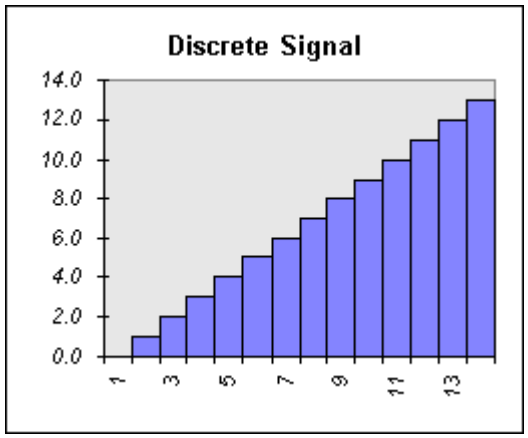
Sensors help translate physical world attributes into values that the computer on a robot can use. The translation produces some sort of output value that the Microcontroller can use. In general, most sensors fall into one of two categories:

1. Analog Sensors
2. Digital Sensors

An analog sensor, such as a CdS cell (Cadmium Sulfide cells measure light intensity), might be wired into a circuit in a way that it will have an output that ranges from 0 volts to 5 volts. The value can assume any

possible value between 0 and 5 volts. An 'Analog Signal' is one that can assume any value in a range. An interesting way to think about this is an Analog Signal works like a tuner on an older radio. You can turn it up or down in a continuous motion. You can fine tune it by turning the knob ever so slightly.





Digital sensors generate what is called a 'Discrete Signal'. This means that there is a range of values that the sensor can output, but the value must increase in steps. There is a known relationship between any value and the values preceding and following it. 'Discrete Signals' typically have a stair step appearance when they are graphed on chart. If you consider a television sets tuner, it allows you to change channels in steps.

For example, consider a push button switch. This is one of the simplest forms of sensors. It has two discrete values. It is on, or it is off. Other 'discrete' sensors might provide you with a binary value. A digital compass, for example, may provide you with your current heading by sending a 9 bit value with a range from 0 to 359. In this case, the Discrete Signal has 360 possibilities.

The most common discrete sensors used in robotics provide you with a binary output which has two discrete states. Much of this article assumes a digital signal to be a binary signal. I will point out exceptions to this rule as we go.

The distinction between Analog and Digital is important when you are deciding which type of sensor you wish to use. Part of this decision depends on the type of resources available on your Microcontroller.

Analog to Digital Conversions

Microcontrollers almost always deal with discrete values. Controllers such as the 68HC11 deal with 8 bit values. An important part of using an Analog Signal is being able to convert it to a Discrete Signal such as a 8-bit digital value. This allows the Microcontroller to do things like compute values and perform comparisons. Fortunately, most modern controllers have a resource called an Analog to Digital converter (A/D converter).

The function of the A/D converter is to convert an Analog signal into a digital value. It does this with a mapping function that assigns discrete values to the entire range of voltages. It is typical for the range of an A/D converter to be 0 to +5 volts.

The A/D converter will divide the range of values by the number of discrete combinations. For example, the table on the right shows 5 samples of an Analog Signal that have been converted into digital values.

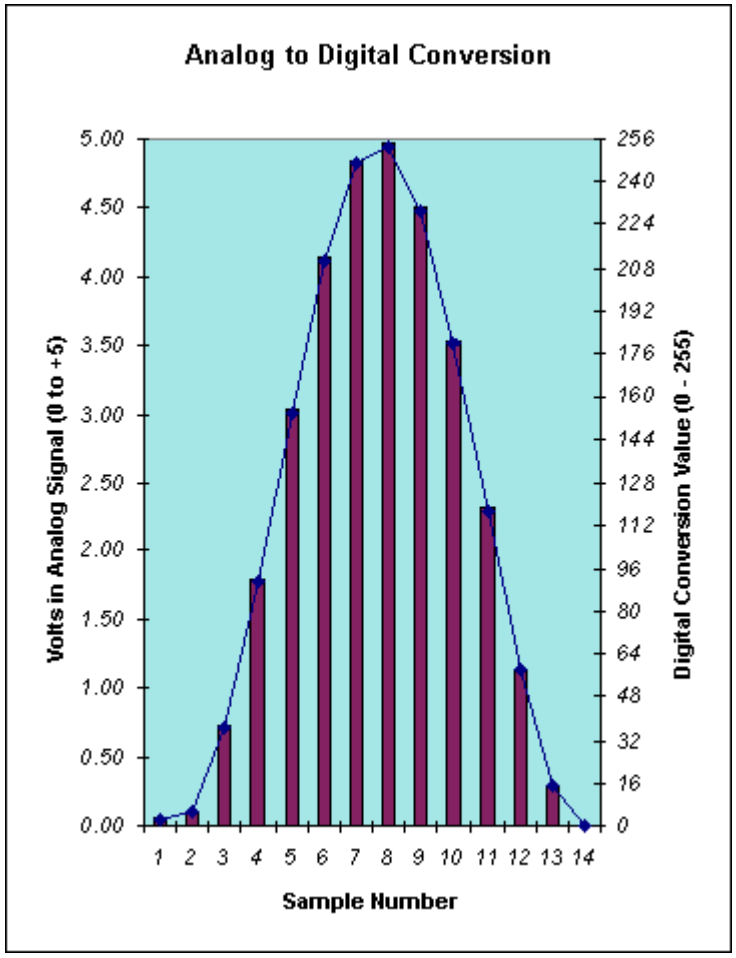
The range of the Analog Signal is 0 to +5 volts. It is a 8-bit A/D converter, which has 256 discrete values. Therefore, the A/D converted divides 5 volts by 256 to yield approximately .0195 volts per unit. The table shows how voltages map to specific conversion values. I have only included the first five, but the table would continue up to conversion value 255.

>= Volts	< Volts	Conversion
0.0000	0.0195	0
0.0195	0.0391	1
0.0391	0.0586	2
0.0586	0.0781	3
0.0781	0.0977	4

The Chart on the left shows the results of the A/D conversions for 14 samples. The sample numbers are shown along the X axis at the bottom. The left hand Y axis indicates the voltage of the Analog sample that was fed into the A/D converter. On the right hand side, the 8-bit value assigned to the conversion is show.

As you can see from the blue line, this was an analog function just like the original Analog Signal graph shown above. The A/D converter has mapped a set of discrete values onto this graph.

There are many types of A/D converters on the market. An important feature is the resolution of the converter. An 8-bit converter is fairly common on Microcontrollers. There are others. A



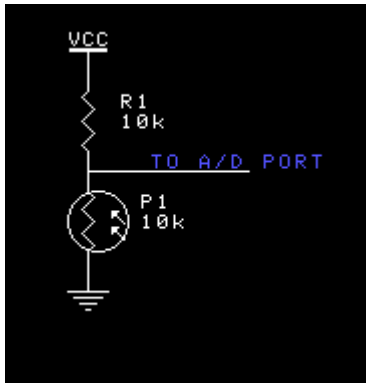
10-bit converter, for example, will divide by 1024 samples. A 16-bit A/D converter can do 65356 discrete values. The resolution required for your application depends on the accuracy your sensor requires. The higher the resolution, the greater the accuracy.

Useful Analog Sensors

Remember, to successfully use an Analog sensor, you need some way to convert the data into a digital form. All of the circuits shown in this section are intended to be connected to a A/D converter port. Many Microcontrollers, such as the 68HC11, have A/D ports built in. Others require that you add an additional support chip, such as the ADC805 or other equivalent chip.

CdS Cells

Cadmium-Sulfide is an interesting compound. Its resistance changes readily when exposed to light energy. Typically, the more light, the lower the resistance. This is useful for measuring the intensity of light.



The CdS cell, shown as P1 in the schematic to the left, has a resistance of 10k in average operating light. I have chosen R1 to have the value of 10k based on this. You should test the CdS cell that you are planning to use to determine its average value. By setting the values close to each other, the average value will be halfway through the range of possible values.

For example, in average light, the CdS cell has resistance of 10k. Using a resistor divider equation, I know that the voltage going to the A/D port will be. V = Vcc * P1 / (P1 + R1) = 5.0 * (10k / (10k + 10k)) = 2.5 volts. Therefore, the A/D port should read around 128 in average light.

CdS cell wiring diagram

Note that as the light increases, the resistance decreases. Assume that it is bright enough for P1 to be only 2k. Vcc * (2k / (2k + 10k)) = 0.83 volts. Using the .0195 voltage value from above, I would expect the A/D conversion result to be approximately 42. Hence, as the light gets brighter, the value from the A/D drops!

In Summary, choosing the value for R1 based on the average reading for the CdS cell will center the 'average' reading at half of Vcc. Doing so allows you to have maximum range on your sensor.

Potentiometers

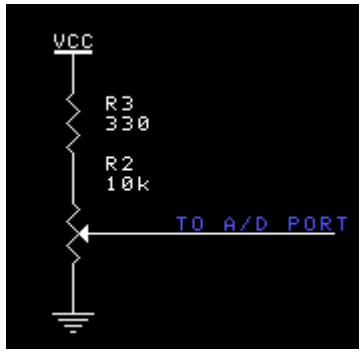
An often overlooked but extremely useful sensor is the good old POT. They are especially useful for making angular measurements, since most pots only turn approximately 270 degrees or so. They are great for determining the angles of a robot arm, for example.

As with the CdS cell, a Potentiometer is a resistive sensor. Almost all resistive sensors are wired in a similar fashion. As you can see by comparing the schematic on the right with the CdS schematic, the key is to make the resistive sensor part of a voltage divider.

The circuit works just like the CdS example. A few things to point out. It is important that the POT be connected to both

Vcc and GND. Otherwise, the divider network is broken and will not function properly. You also want to insure your POT is large enough not to allow too much current to flow. A POT with a resistance of > 1k should be fine. A POT with > 100k of resistance is also a good choice, since the amount of current consumed by the circuit is extremely low.

Notice in this circuit the current limiting resistor R3. This resistor is there to handle the case when the sweep on the POT is turned all the way to the 'top' position. Without it, a large amount of current could flow if the output was accidentally connected to the wrong port, or if the A/D port on your Microcontroller was by-directional.



Potentiometer Sensor

Wiring Diagram

Using the values in the shown schematic, you can calculate what the voltage ranges the pot will allow. With the sweep all the way to the 'top', the value for R2 at the sweep is 10k. The voltage drop across R2 = Vcc * (R2 / (R2 + R3)) = 5.0 * (10k / (10K + 330)) = 4.84 volts. Thus, the highest digital value will be 4.84 / 0.0195 = 248. Actually, it will be 247 since the A/D conversions are zero based. The lowest value should be zero, since with the sweep all the way to the bottom, the A/D port will be connected to GND. Thus, the limiting resistor has reduced the useful range of the POT.

To increase the range, you can increase the value of R2. For example, using a 100K pot means 5.0 * (100k / (100k + 330)) = 4.98 volts. Thus, 4.97 / .0195 = 255, which will be 254 when adjusted for the zero based conversion.

There are two types of potentiometers on the market. Audio and Linear. A Linear pot changes its value at a linear rate. There is an easy mathematical relationship between the angular position and the resistance.

An 'Audio Taper' or 'Audio' pot changes its value on a logarithmic scale. These are not well suited as positional sensors.

Useful Digital Sensors

There are many different types of digital input sensors. Many of them are wired in the same form, which uses a pull-up resistor to force the line high, and to limit the amount of current that can flow. If you have questions about pull-ups and current limiting resistors, you might like to check out The Basics - Very Basic Circuits for more information about these subjects.

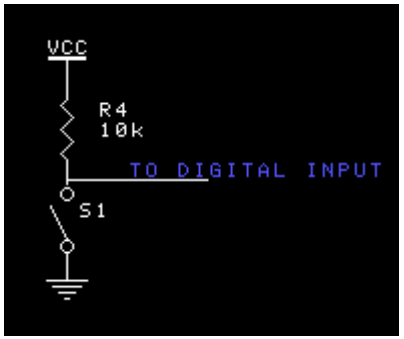
Switches

One of the most basic of all sensors is a simple switch. Switches are used in bumper sensors, to detect limits of motion, for user input, and a whole host of other things.

Switches come in two types: normally open (NO) and normally close (NC). Many microswitch designs actually have one common terminal, and both a NO terminal and a NC terminal. If you have a switch that has three terminals, chances are this is the arrangement.

The wiring diagram for a switch is very easy. I recommend using a NO switch to limit the amount of power consumed. With a 10k pull-up, the amount of current is small, but many switches can add up to some noticeable power.

Important points are to use a pull-up resistor that doubles as a current limiting resistor. In the event that your program accidentally switches the input port to an output port, having the current limiting resistor will keep from frying your Microcontroller.

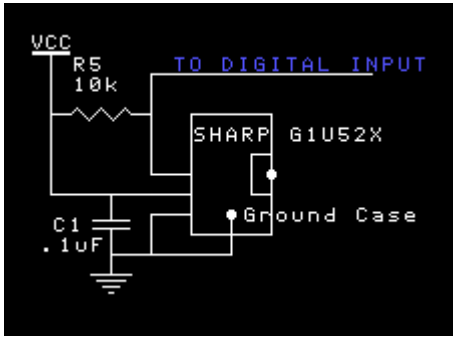


Infrared Detectors

Infra-red detection is a common thing to add to a robot. It allows the robot to determine when it has come in close proximity to an object without coming into physical contact. A typical way of detecting infra-red is to use a Sharp G1U52X module. To learn more about this, check out Implementing Infrared Object Detection.

The basic wiring diagram for the Sharp module is shown on the right. The connections are to power, ground, and the output signal. The output from the sharp detector is a digital signal.

Notice that R5 acts as a pull-up resistor, similar to other digital inputs. Capacitor C1 acts as a bypass capacitor. Another unusual connection is between ground and the case. Most of the Sharp modules are intended to be mounted on a circuit board. It expects the case to be grounded. Be sure to make a electrical connection between ground and the case by soldering a wire directly to the metal housing.



Wiring a Sharp IR detector

Summary

Sensors usually output one of two types of signal. An Analog signal or a Discrete signal. Microcontrollers usually deal with discrete or digital signals. An Analog to Digital converter allows the output of an analog device to be used by a Microcontroller. Many Microcontrollers have A/D converters built in.

Interfacing sensors is fairly straight forward. You need to be aware of what type of output a sensor provides. You also need to be careful not to create a path that allows too much current to flow. Current limiting resistors are important in interfacing Microcontrollers to sensors.

What else do you want to know? Send me mail and ask away. Kevin Ross